University of Cyprus

Machine Learning methods for Boolean Networks

Costas Frantzides

Advisor: Dr Dimopoulos Yiannis
Department of Computer Science, University of Cyprus

Introduction and Motivation

University of Cyprus

Need for a clear exploration and evaluation of available tools

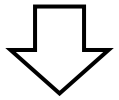
A clear connection is needed between modelling, simulation, analysis and reconstruction

- This thesis examines these tools and combines them in a configurable pipeline

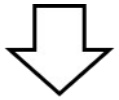
Outline

University of Cyprus

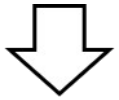
1. What is a Boolean Network?



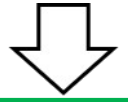
2. Computational Tools for interacting with BNs



3. **Sketches Pipeline overview**



4. **Sketches Experiment and Evaluation**



5. Conclusion and Future Directions

1. What is a Boolean Network?

University of Cyprus

Variables: $X = \{x_1, x_2, \ldots, x_n\}$, each $x_i \in \{0, 1\}$

Functions: $F = \{f_1, f_2, \ldots, f_n\}$, each $f_i : \{0,1\}^n \rightarrow \{0,1\}$

```
f_A = C
f_B = A AND C
f_C = NOT B
```

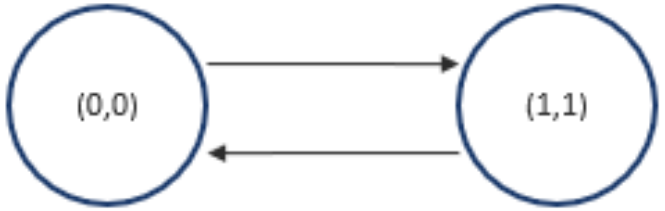
State: $s = (x_1, x_2, \ldots, x_n) \in \{0,1\}^n$; 2^n states in total

```
(A=0 , B=1 , C=0)
```

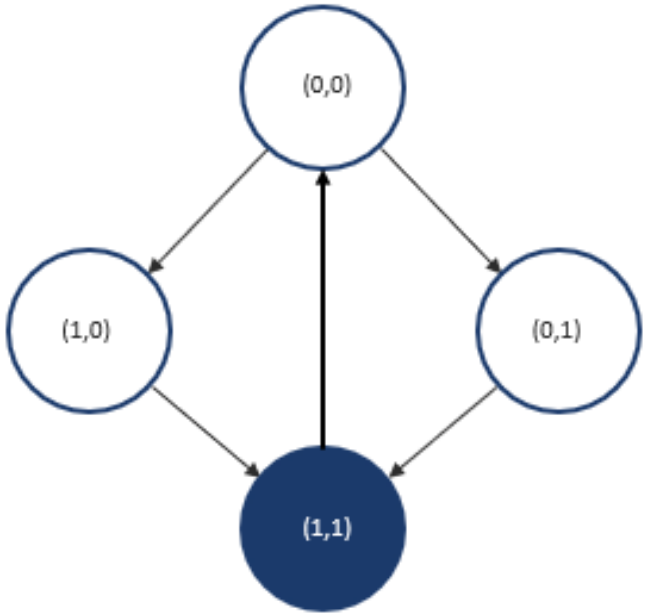
1. What is a Boolean Network?

University of Cyprus

Synchronous update:
All variables update simultaneously
Unique next state
Deterministic system



Asynchronous update:
One variable updates at a time
Multiple possible next states
Non-deterministic system



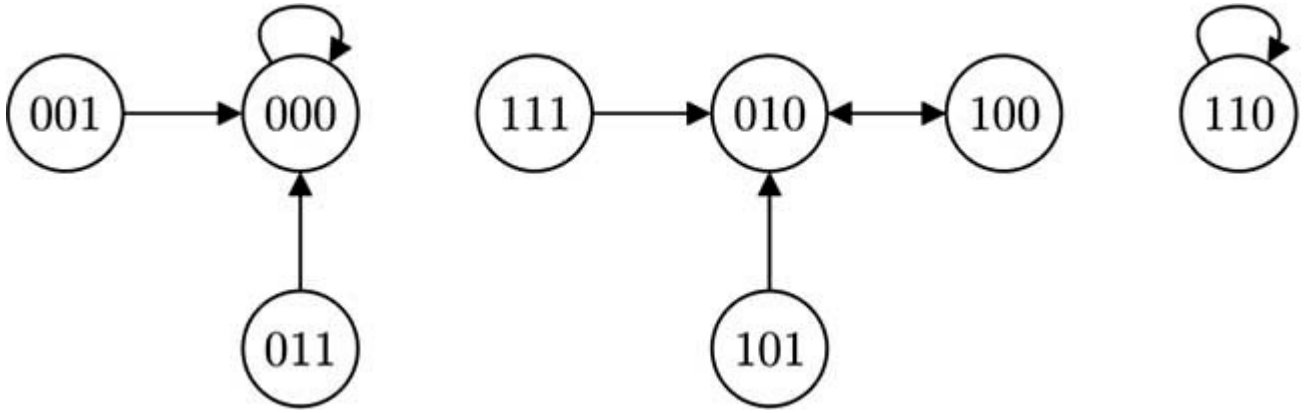
University of Cyprus

Transition graph

A directed graph where:

— each node is a state $s \in \{0,1\}^n$

— each edge $s \rightarrow s'$ is a transition under F



For n=3 variables: 8 states, 8 edges (synchronous update)

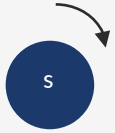
1. What is a Boolean Network?

University of Cyprus

Fixed Point

F(s) = s

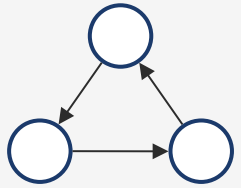
The system remains in the same state



Cyclic Attractor

$s_0 \rightarrow s_1 \rightarrow \ldots \rightarrow s_k \rightarrow s_0$

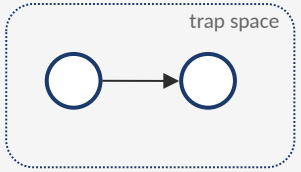
A repeating sequence of states



Trap Space

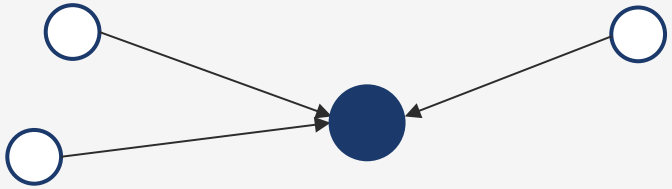
A subspace that no trajectory can leave

Every attractor lies within a minimal trap space



Basin of Attraction

Initial states leading to the same attractor; basin size measures robustness



University of Cyprus

Structural Properties of Boolean Functions:

Monotonicity

A variable has a consistent effect:
activating
inhibiting
no effect

Example:
f_B = A AND C
A and C are positive inputs

Essentiality

A variable is essential if changing it can change the function output.

Example:
f_C = NOT B
B is essential for C

Canalization

A specific input value determines the output regardless of other variables.

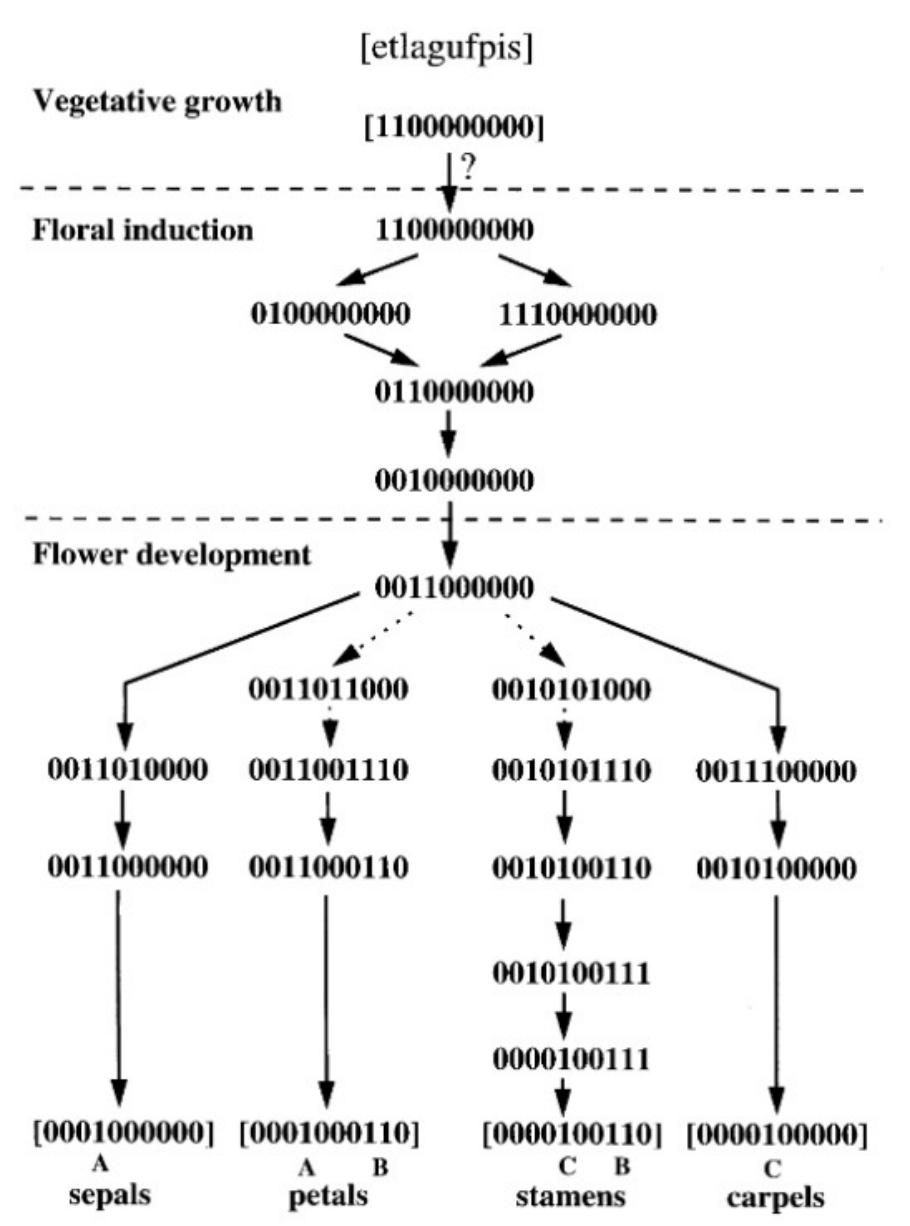
Example:
f = A OR B
If A=1, then f=1

University

Flower development Arabidopsis thaliana

Modelling cell-fate decisions in plant development.

- Attractors correspond to different biological states.



1. What is a Boolean Network?

University of Cyprus

Embryonic development of Drosophila

A classic example of gene regulation.

- Shows how simple logical rules reproduce developmental patterns.

T-LGL leukaemia survival network

Cell survival/apoptosis network.

- Useful for studying interventions and potential therapeutic targets.

University of Cyprus

BoolForge

- Language: Python
- Purpose: Generate, simulate and analyse Boolean networks

- Main use: Create networks with controlled structural properties

Network Generation
Generate networks with specified parameters

Simulation
Simulate trajectories synchronously or asynchronously

Attractor Analysis
Identify fixed points and cyclic attractors

Python API
Easy integration into automated workflows

University of Cyprus

Boolnet

- **Language: R package**
- **Purpose: Simulation, analysis and basic reconstruction of Boolean networks**
- **Community: Widely used in systems biology**

Simulation
Synchronous and asynchronous simulation

Attractor Identification
Identify and classify attractors

Basin Analysis
Calculate basins of attraction

Perturbation Analysis
Robustness under perturbations

Inference from Time-Series
Reconstruction from experimental data

2. Computational Tools for interacting with BNs

University of Cyprus

bioLQM

- **Language: Java**
- **Purpose: Qualitative logical modelling and property extraction**
- **Capability: Interoperability between formats and efficient structural analysis**

Format Conversion
SBML, .bnet, .aeon and others

Fixed Point Computation
Fast fixed-point computation

Trap Space Analysis
Compute minimal trap spaces

Scalability
Efficient for large networks

University of Cyprus

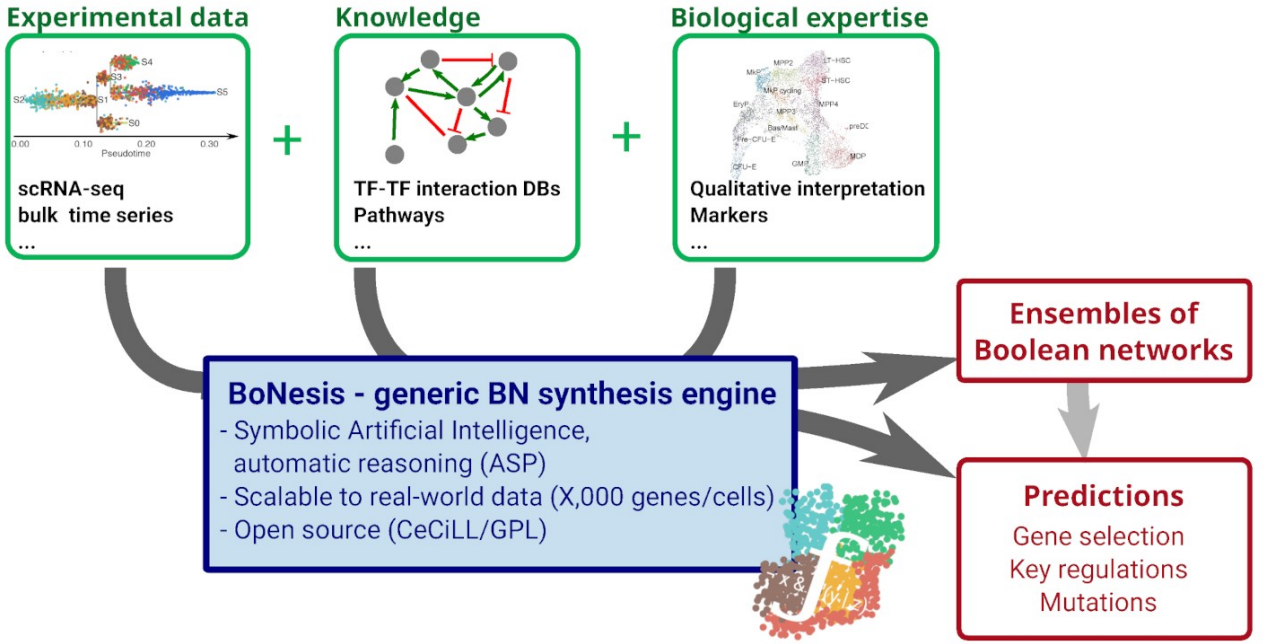
BoNesis

Language: Python

Purpose: Synthesis and reconstruction of Boolean networks

Mechanism: Answer Set Programming (ASP)

- **Core idea: Find networks satisfying structural and dynamic constraints**



University of Cyprus

BoNesis

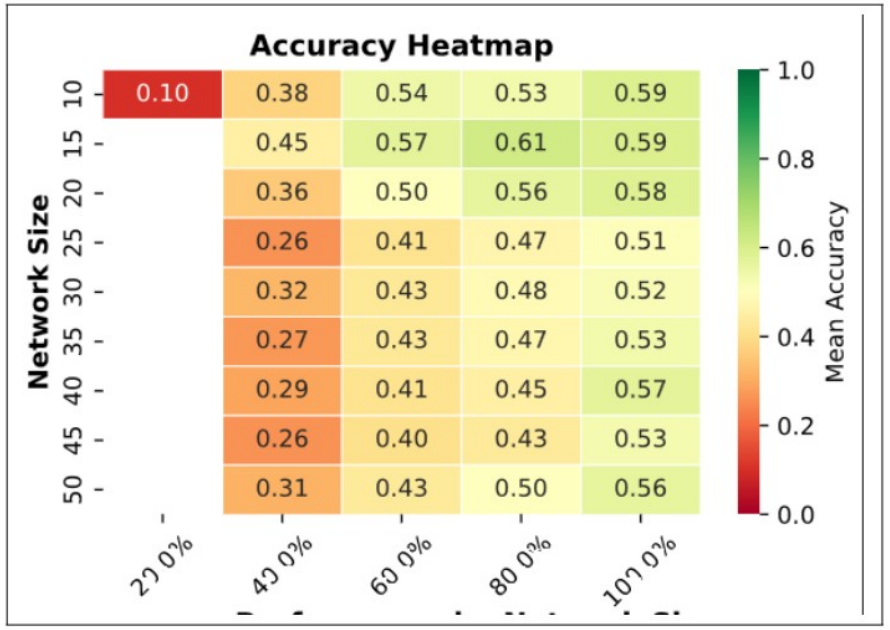
Evaluate reconstruction using observations at different data-availability percentages

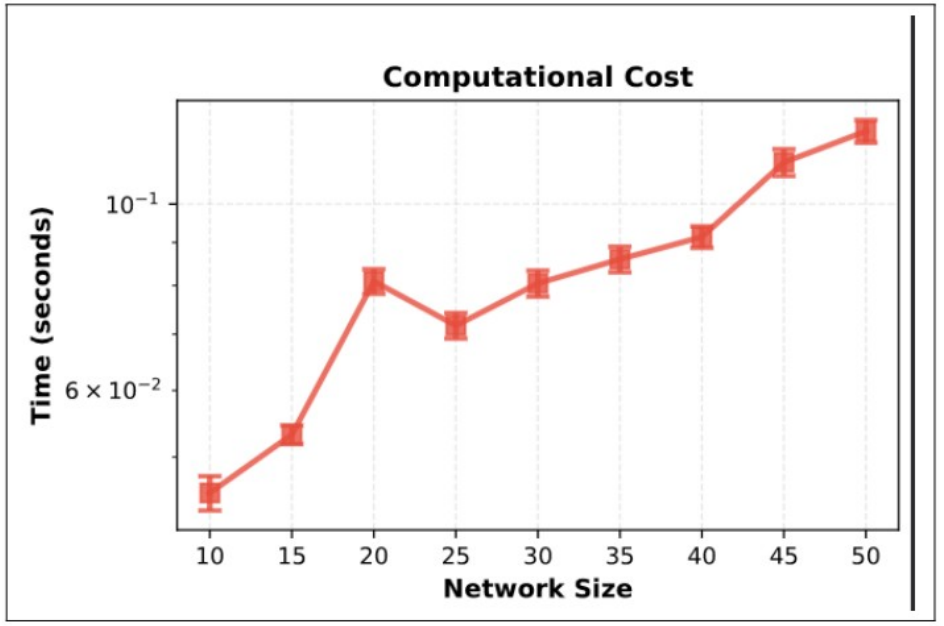
- Compare reconstructed networks against a known ground-truth network

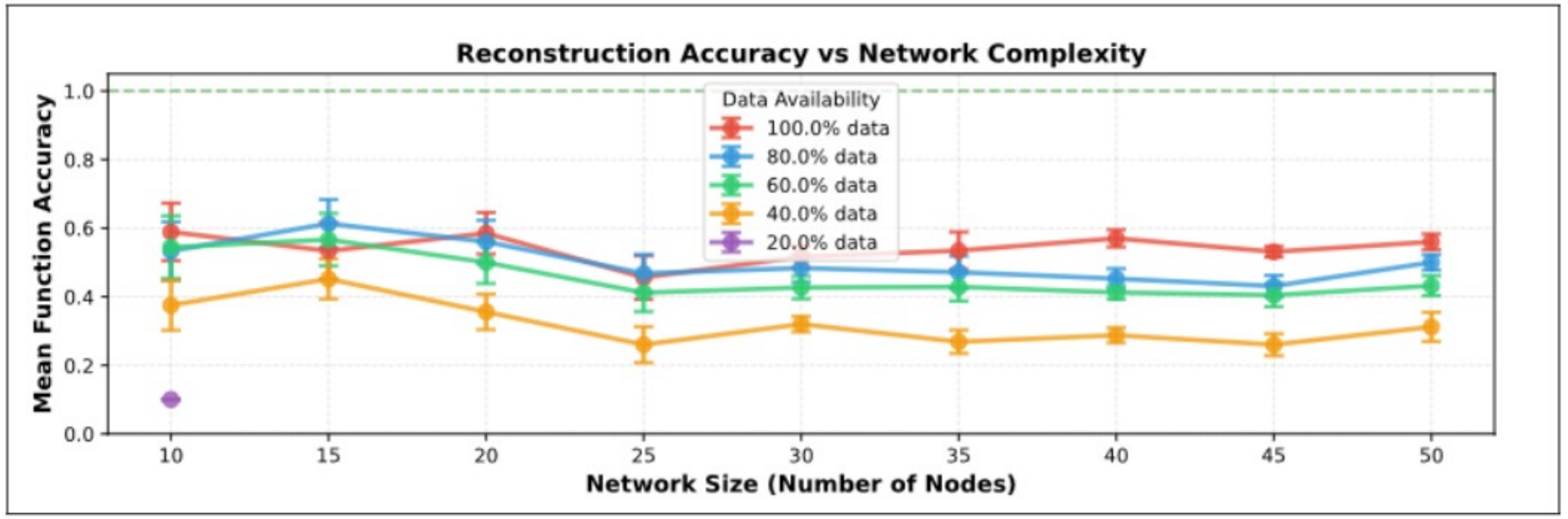
Parameter	Symbol	Value / Range
Network size (nodes)	n	*[5,10,...,50]*
In-degree	k	2
Data fractions	f	*20%, 40%, ..., 100%*
Replicates per condition	R	*100*
Candidate networks checked	N_{sol}	*10*
Trajectory length	T	*100 steps*
Reconstruction timeout	τ	*100 seconds*

2. Computational Tools for interacting with BNs

University of Cyprus

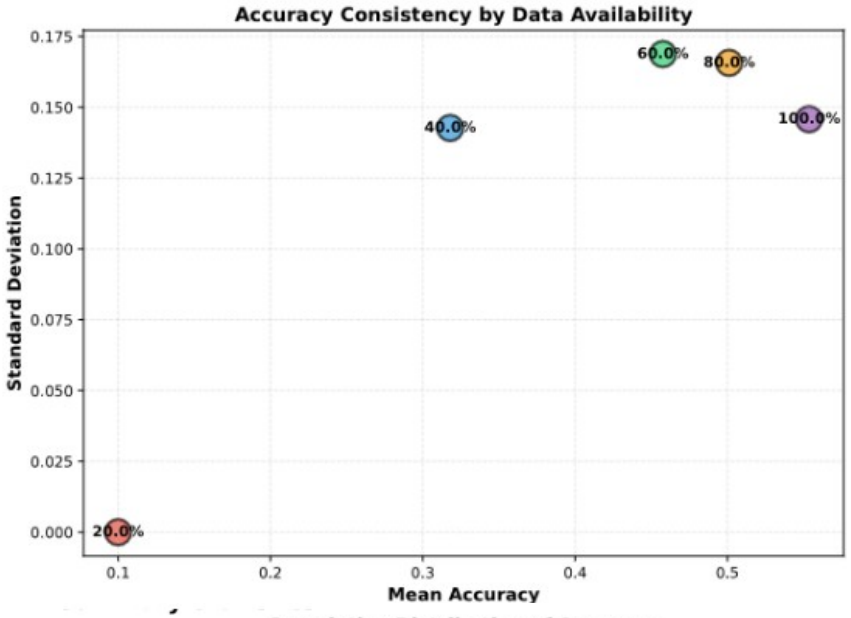


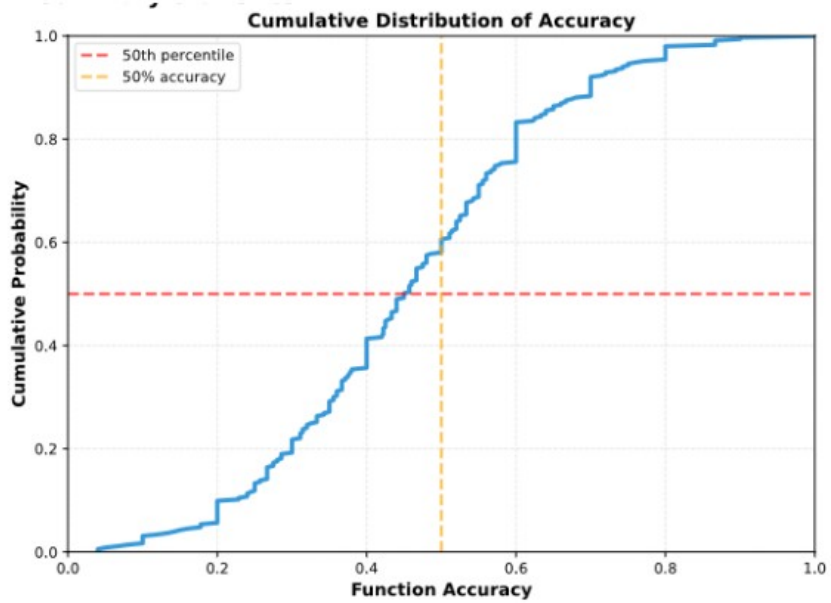


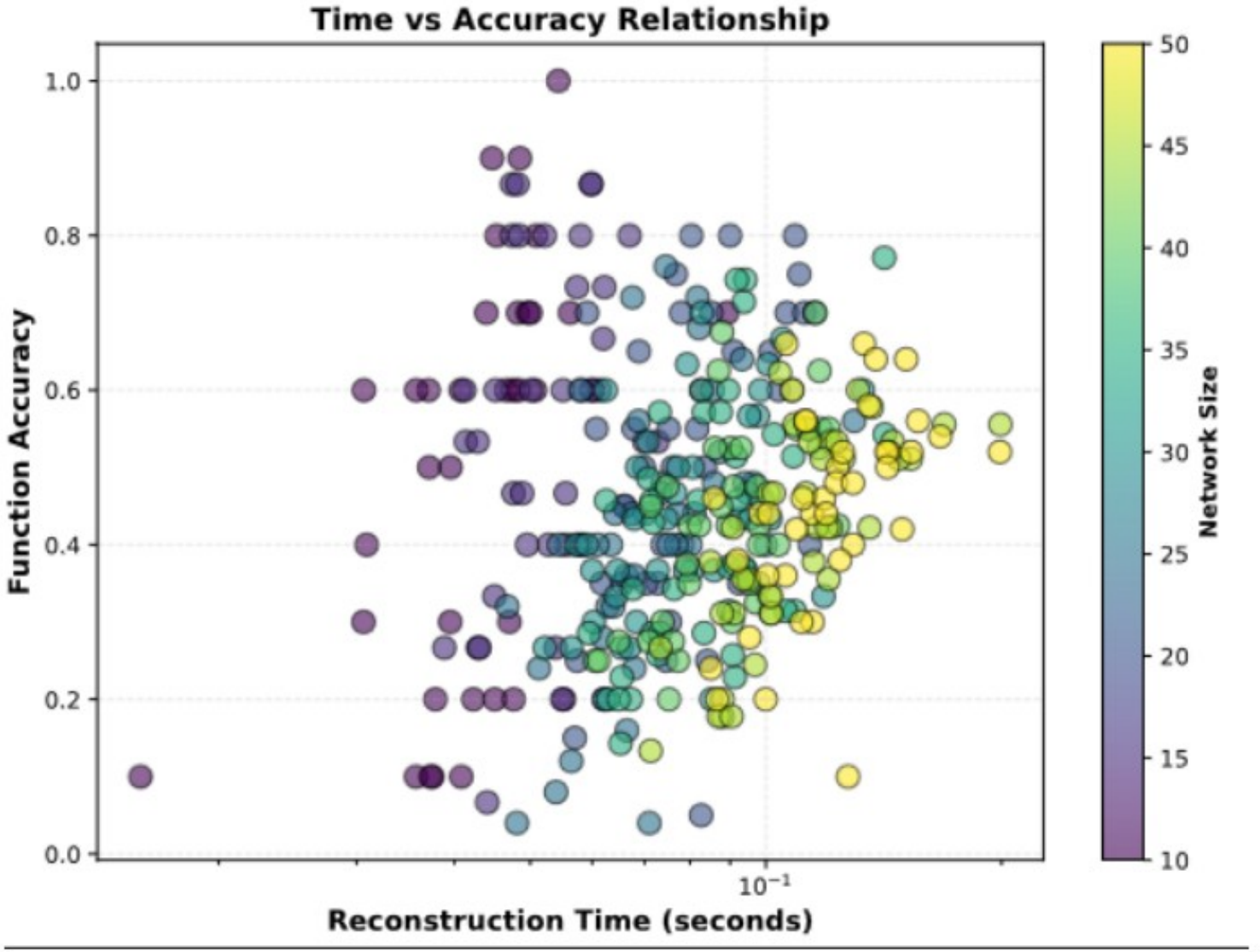


2. Computational Tools for interacting with BNs

BoNesis



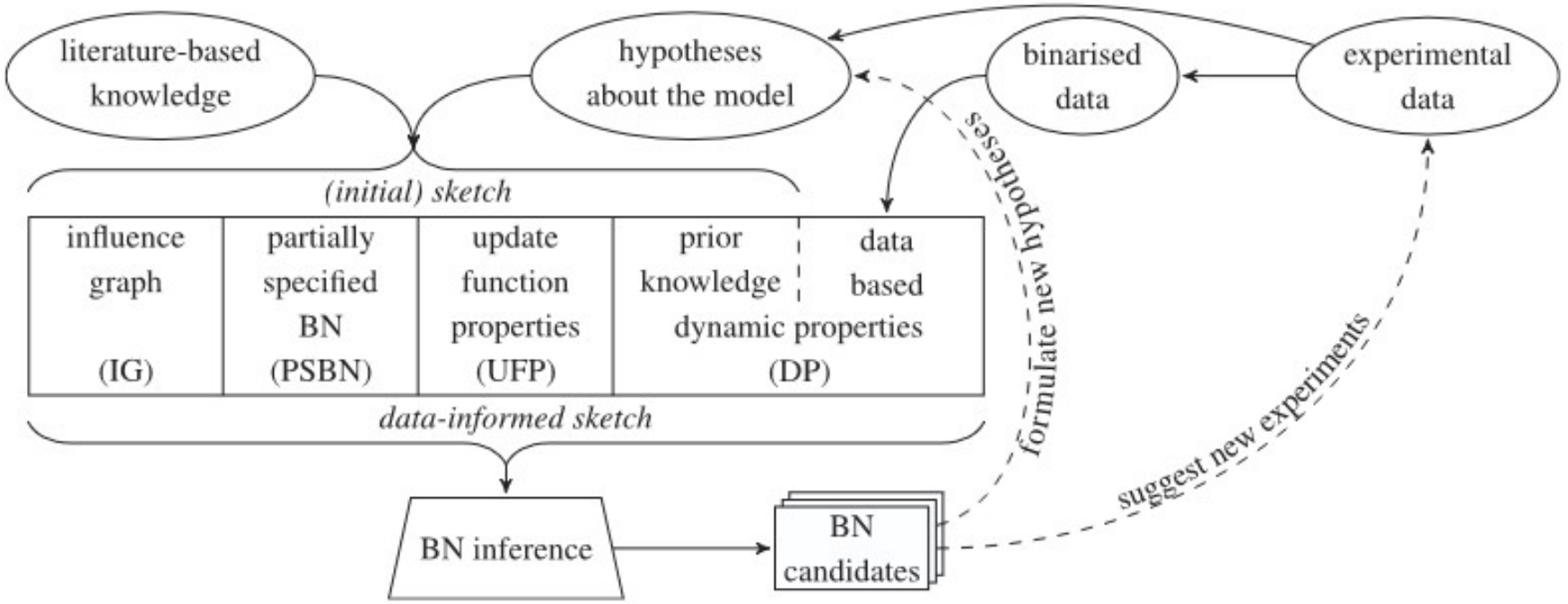




University of Cyprus

Sketches

Language: Rust
Purpose: Reconstruct Boolean networks through symbolic inference and model checking
Capabilities: Combine structural and dynamic constraints (HCTL), find all compatible networks, and support attractors, fixed points and temporal properties



University of Cyprus

Sketches

```
#  Model
# Partially specified update functions
$x1: x2 & x4            # fully specified
$x3: f(x1, x5)          # fully unknown function over x1,x5
$x4: x1 & g(x5,x2)      # partially known
$x5: ?                  # entirely free
```

Monotonicity

```
x7 -> x2
# x7 activates x2

x1 -| x3
# x1 inhibits x3

x6 -? x1
# unknown sign
```

Essentiality

```
$x9: f_x9(x4, x5)

# f_x9 depends
only
# on essential
regulators
# x4 and x5
```

Canalization

```
$x9: !x7 |
g_x9(x6, x19)

# x7=1 forces x9=0
# remaining
uncertainty:
# g_x9(x6,x19)
```

University of Cyprus

Sketches

Γραμματική HCTL

$$\varphi ::= 0 \mid 1 \mid p \mid \mathbf{x} \mid \neg\varphi \mid \varphi \wedge \varphi \mid EX\varphi \mid E[\varphi U \varphi] \mid A[\varphi U \varphi] \mid \mathbf{@x\varphi} \mid \mathbf{\downarrow x.\varphi} \mid \mathbf{\exists x.\varphi} \mid$$

p: atomic proposition / network variable
x: state variable

*Μπορούν να χρησιμοποιηθούν επίσης οι συντακτικές συντομογραφίες για τα **EF**, **AF**, **EG**, **AG** και **AX** καθώς και άλλες συντομογραφίες πρώτης τάξης και προτασιακής λογικής, όπως* $\vee$, $\rightarrow$, *κ.λπ.*

University of Cyprus

Sketches

Bind operator: $\downarrow x \ \varphi$

- Assigns the name x to the current state so it can be referenced later.

Existential state operator: $\exists x \ \varphi$

- There exists a state in the state space for which φ holds.

Jump operator: $@x \ \varphi$

- Moves evaluation of φ to the state stored as x.

University of Cyprus

Sketches

Property	General HCTL form	Meaning
Trap space	3{x}: @{x}: (p & AG p)	There exists a state satisfying partial state pattern p, and from that state, p always holds in the future. Once entered, the pattern is preserved.
Attractor	3{x}: @{x}: (ψ & AG EF ψ)	There exists a state satisfying ψ, and from every future reachable state, ψ can be reached again. This expresses recurrence.
Fixed point	3{x}: @{x}: (ψ & AX ψ)	There exists a state matching ψ, and every immediate successor also matches ψ. If ψ is a full state, this means the state maps to itself.
Clamped fixed point	(3{c}: @{c}: AG κ) => (3{x}: @{x}: (AG κ & ψ & AX ψ))	If the original network admits a region where the clamped condition κ always holds, then inside that region there exists a fixed point matching ψ.
Simple observation	3{x}: @{x}: ψ	There exists at least one state in the state space that satisfies the observation pattern ψ.
Reachability constraint	3{x}: @{x}: (ψ1 & EF ψ2)	There exists a state satisfying ψ1 from which some state satisfying ψ2 is eventually reachable.
Long reachability chain	3{x}: @{x}: (ψ1 & EF(ψ2 & EF(ψ3 & EF ψ4)))	There exists a path that can reproduce a sequence of observations: first ψ1, then eventually ψ2, then eventually ψ3, then eventually ψ4.
One-step reachability	3{x}: @{x}: (ψ1 & EX ψ2)	There exists a state satisfying ψ1 with an immediate successor satisfying ψ2

3. Sketches Pipeline overview

University of Cyprus

General idea

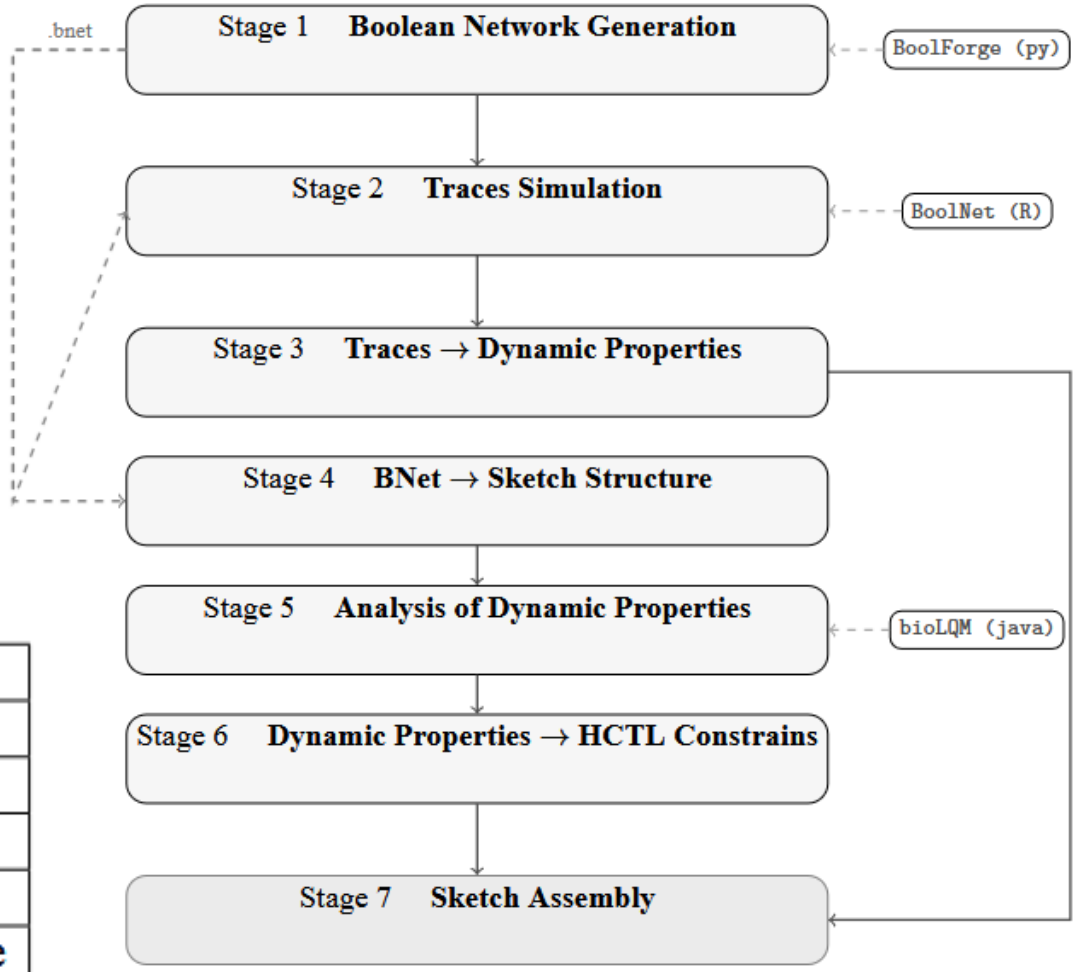
Configurable generation of synthetic sketches
Integration of tools into a unified pipeline

- Selection of the amount and type of information in the final sketch

Purpose

Experimental study of the Sketches framework
Tool for generating sketches resembling real systems

- Evaluate which information best supports realistic reconstruction



Stage	Tool	Role
Generation	BoolForge	Create ground-truth network
Simulation	BoolNet	Generate traces
Analysis	bioLQM	Compute fixed points / trap spaces
Reconstruction	Sketches	Infer consistent networks
Comparison	BoNesis	Alternative constraint-based inference

Table 6.1: Tools used in the experimental pipeline

3. Sketches Pipeline overview

University of Cyprus

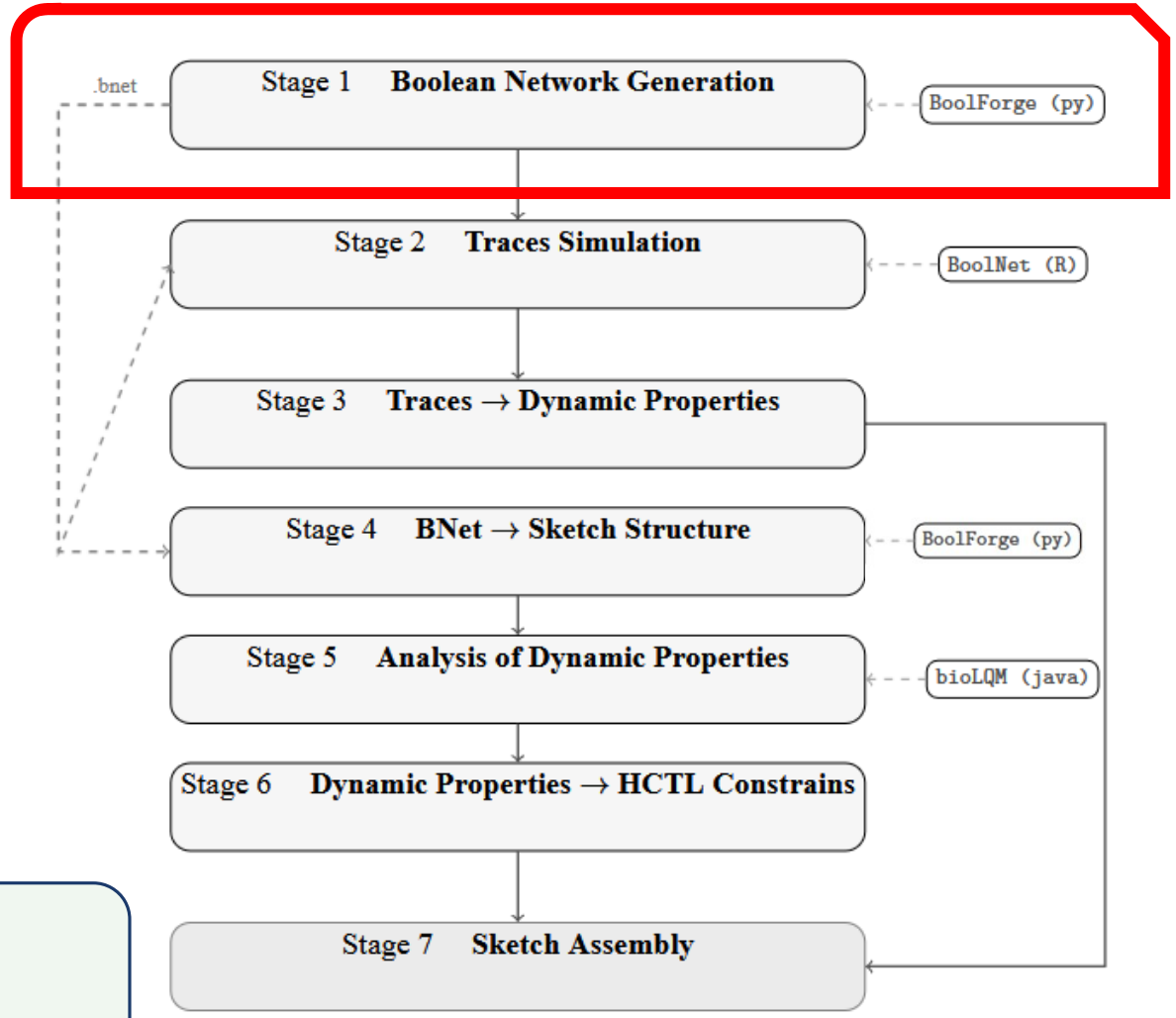
Stage 1 - Boolean Network

INPUT

BoolForge config

Config parameters

Configuration key	Purpose
`output`	Path of the generated `.bnet` file.
`N`	Number of variables in the generated Boolean network.
`n`	Maximum in-degree used when constructing the regulatory graph.
`indegree_distribution`	Distribution governing regulator counts.
`bias`	Bias used during rule generation.
`depth`	Logical depth of generated update functions.
`NO_SELF_REGULATION`	Prevents variables from regulating themselves.
`STRONGLY_CONNECTED`	Requests generation of a strongly connected interaction graph.
`AT_LEAST_ONE_REGULATOR_PER_NODE`	Ensures that each variable has at least one regulator.
`acyclic`	Requests an acyclic interaction structure.
`acyclic_method`	Selects the strategy used to construct the acyclic graph.
`rng`	Random seed for reproducible generation.



OUTPUT

A Boolean network in .bnet format

```
targets, factors
x1, x16 | x3
x2, x10 & !x9  # Each line defines
x3, x15 | x2  # an update rule for a variable
...
```

3. Sketches Pipeline overview

University of Cyprus

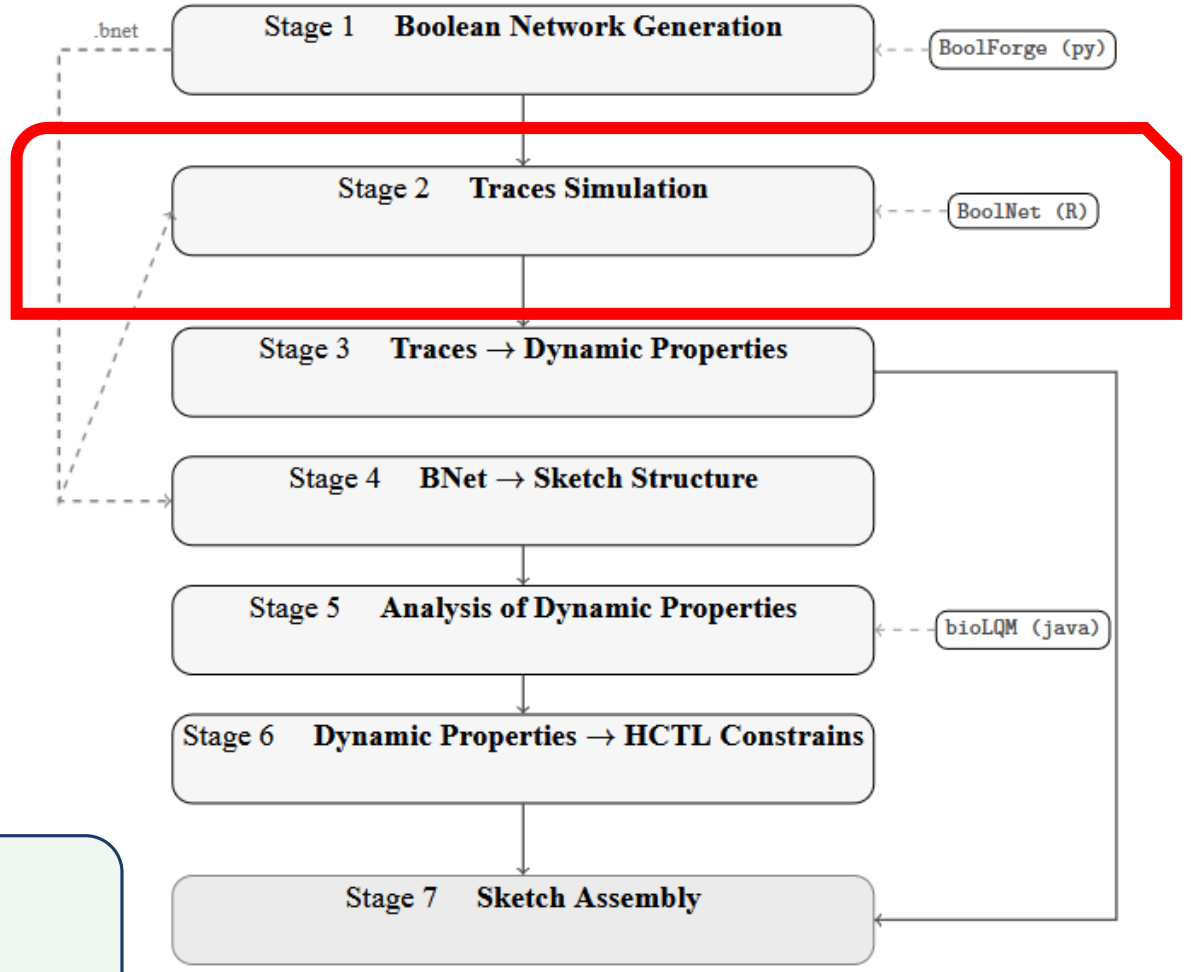
Stage 2 – Trace Simulation

INPUT

Bnet from Stage 1 + Stage 2 configuration

Config parameters

Configuration key	Purpose
num_traces	Number of trajectories to generate.
num_steps	Number of states recorded in each trajectory.
update_type	Choice of update semantics, e.g. synchronous or asynchronous.
noise_level	Noise level used during simulation.
seed	Random seed for reproducible simulations.
output_dir	Directory in which trace files are written.
output_prefix	Prefix used for naming trace files.
output_suffix	Suffix used for naming trace files.
write_trajectory_header	Controls whether trace files contain an explicit header.
write_genes_file	Controls whether a file containing the variable names is also produced.



OUTPUT

trace files /
state trajectories

```
x1 x2 x3 x4 ... x20
0  0  0  1  ... 1
0  0  0  1  ... 1
1  0  1  0  ... 0
...
```

3. Sketches Pipeline overview

University of Cyprus

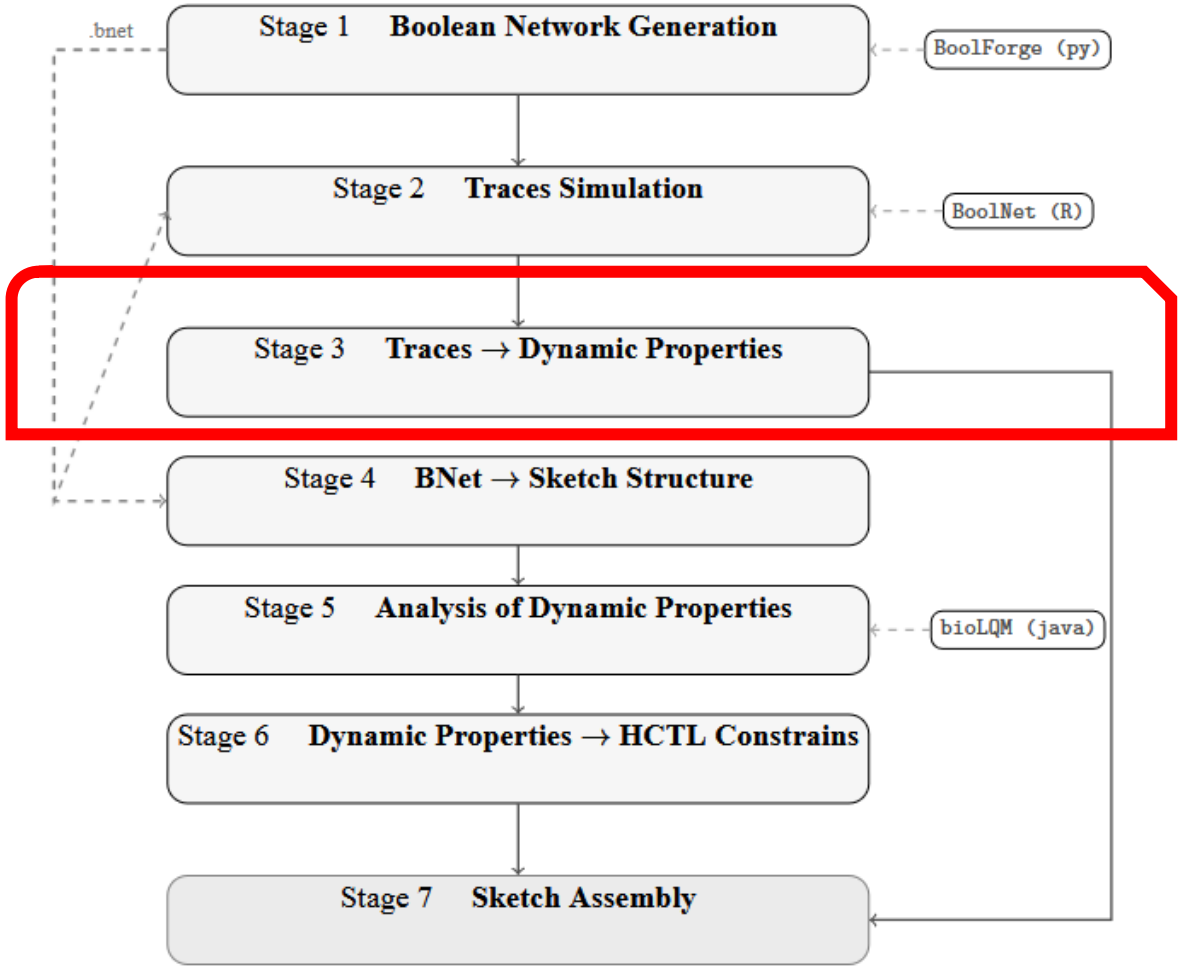
Stage 3 - Trace Conversion to Dynamic Properties

INPUT

Trace files from Stage 2 + Stage 3 configuration

Config parameters

Configuration key	Purpose
`traces_dir`	Directory containing the traces produced in Stage 2.
`output`	Path of the generated sketch-property snippet.
`genes`	Optional path to the file listing the variables.
`trace_glob`	Pattern used to select trace files.
`pair_mode`	Strategy used to derive reachability constraints from traces.
`keep_percent`	Fraction of generated properties retained after sampling.
`partial_state_size`	Number of variables retained when projecting states.
`partial_state_mode`	Strategy for selecting the retained variables.
`seed`	Random seed for property sampling or variable selection.
`compress_stutter`	Removes consecutive duplicate states prior to analysis.
`property_prefix`	Naming prefix for reachability properties.
`fixed_point_prefix`	Naming prefix for singleton-tail recurrence candidates.
`cycle_prefix`	Naming prefix for cycle candidates.
`cycle_lengths`	Cycle lengths considered when detecting repeated patterns.



3. Sketches Pipeline overview

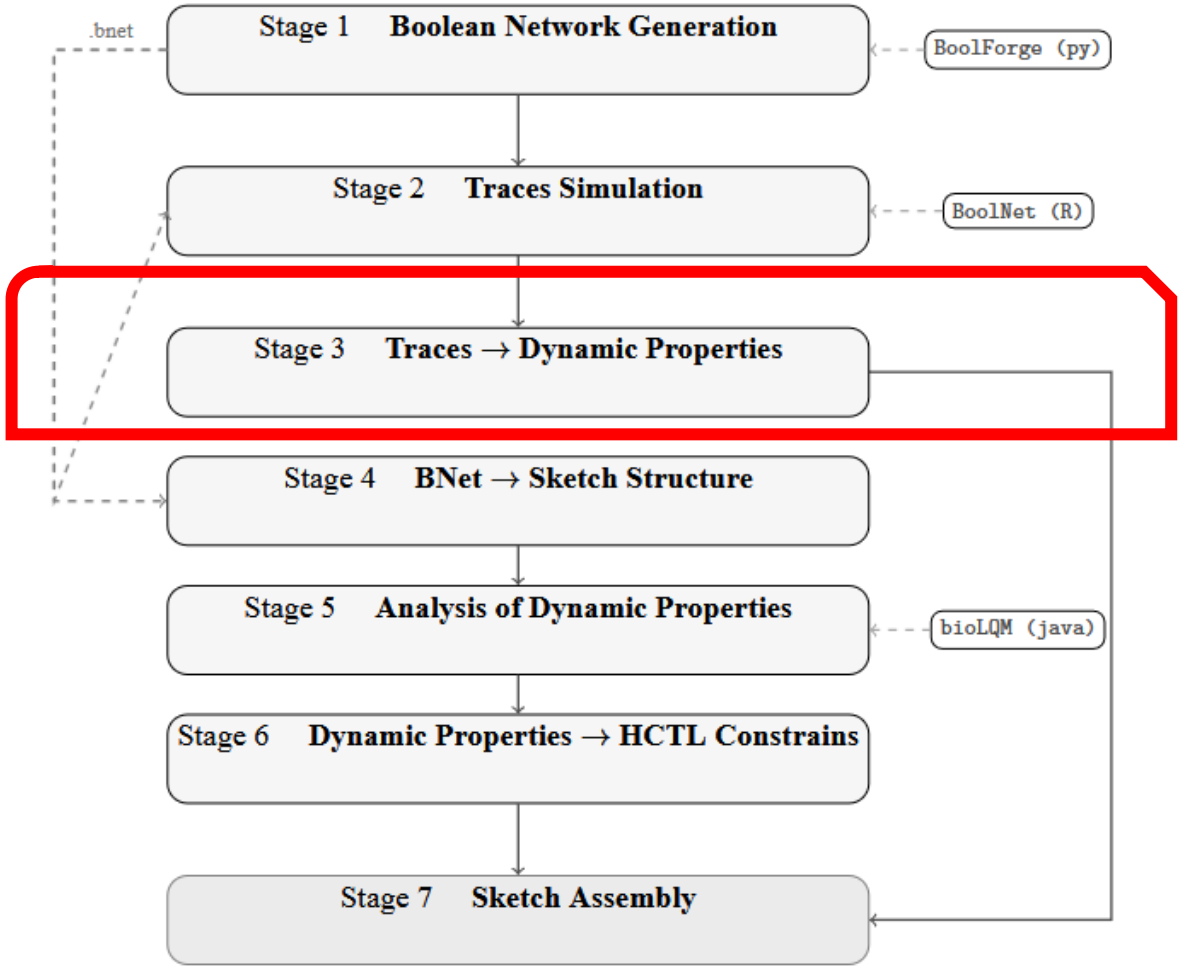
University of Cyprus

Stage 3 - Trace Conversion to Dynamic Properties

OUTPUT
A .aeon file containing HCTL constraints:

```
#! dynamic_property: reachability_1:
#`3{x}: ( @{x}: (
#    (x6 & x12 & ~x16 & x18)
#    & EF((~x6 & x12 & ~x16 & ~x18))
#) )`#

#! dynamic_property: reachability_2:
#`3{x}: ( @{x}: (
#    (~x6 & ~x10 & ~x16 & x18)
#    & EF((~x6 & x10 & ~x16 & ~x18))
#) )`#
...
```



3. Sketches Pipeline overview

University of Cyprus

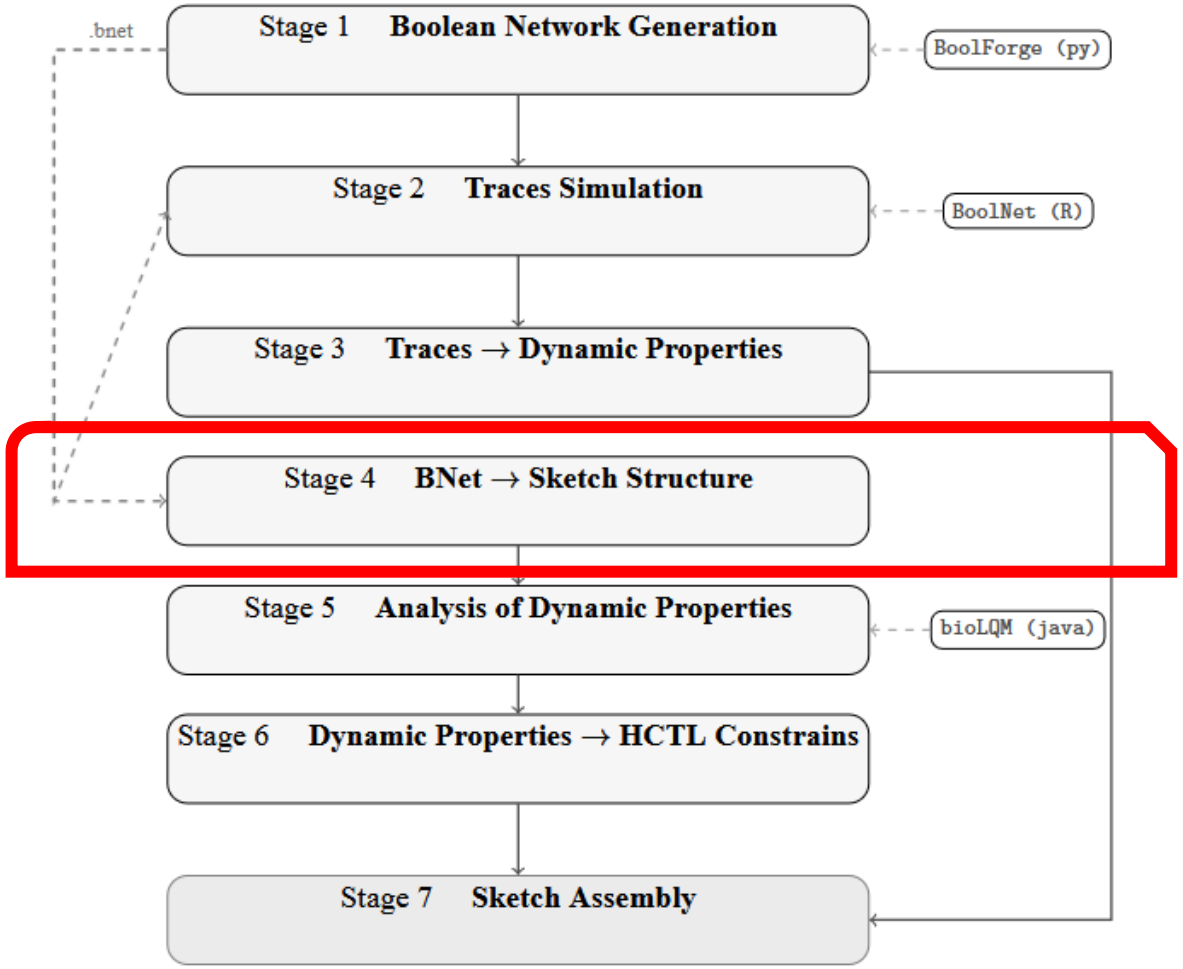
Stage 4 - Structural Sketch Construction

INPUT

Bnet from Stage 1 + Stage 4 configuration

Config parameters

Configuration key	Purpose
reveal_functions_percent	Fraction of target functions for which regulator support is revealed.
reveal_regulators_percent	Fraction of regulators revealed for support-revealed targets.
reveal_exact_functions_percent	Fraction of target functions copied exactly into the sketch.
seed	Random seed controlling reveal choices.
edge_op	Default edge symbol used for unconstrained interactions.
hidden_policy	Policy for representing targets whose structure remains hidden.
infer_monotonicity_for_exact	Enables sign inference for exactly revealed rules.
infer_essentiality	Enables detection of essential and non-essential regulators.
apply_essentiality_to_symbolic_supports	Restricts symbolic supports to essential regulators only.
annotate_essentiality_comments	Adds essentiality annotations to the structural output.
infer_canalization_for_exact	Enables canalization analysis for exactly revealed rules.
apply_canalization_templates	Replaces generic symbolic rules by canalization-aware templates when possible.
annotate_canalization_comments	Adds canalization annotations to the structural output.



3. Sketches Pipeline overview

University of Cyprus

Stage 4 - Structural Sketch Construction

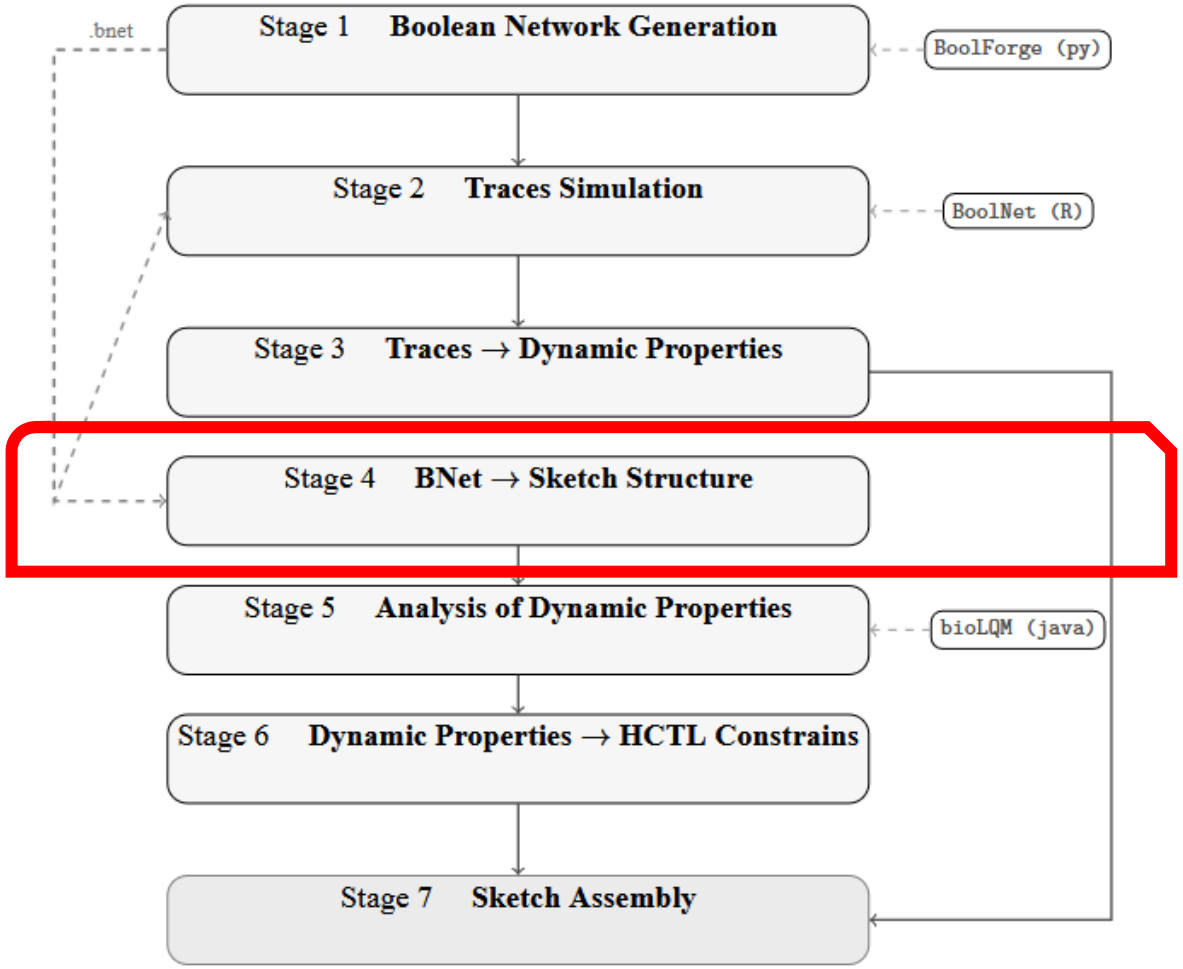
OUTPUT

```
## MODEL
x10 -> x2
x9  -| x2

$x2: x10 & g_x2(x9)
x7  -| x9
x6  -?? x9
x19 -?? X9

# canalization x9:
# x7=1 => x9=0
# essentiality x9:
# essential=x7,x6,x19;

 $x9: !x7 | g_x9(x6, x19)
```



3. Sketches Pipeline overview

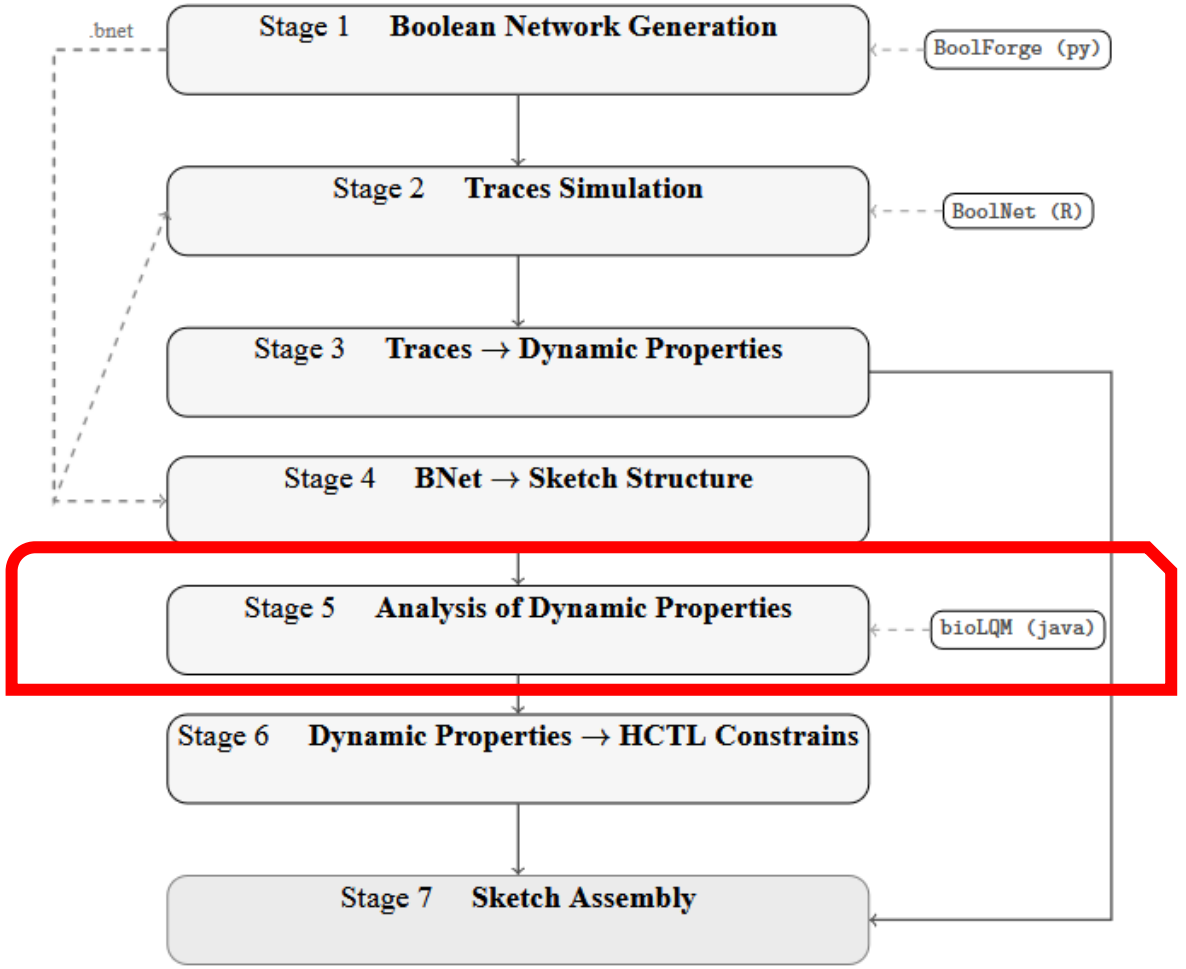
University of Cyprus

Stage 5 - Boolean Network

INPUT
Bnet from Stage 1 + Stage 5 configuration

Config parameters

Configuration key	Purpose
`bnet`	Path to the `.bnet` model produced in Stage 1.
`fixpoints_output`	Path of the file containing fixed-point analysis results.
`trapspaces_output`	Path of the file containing trap-space analysis results.
`biolqm_cmd`	Command used to invoke `bioLQM`.
`java_cmd`	Java executable used when required by the analysis back-end.
`biolqm_jar`	Optional path to the `bioLQM` archive.
`skip_fixpoints`	Disables fixed-point analysis.
`skip_trapspaces`	Disables trap-space analysis.



3. Sketches Pipeline overview

University of Cyprus

Stage 5 - Boolean Network Ge

OUTPUT

raw bioLQM analysis results

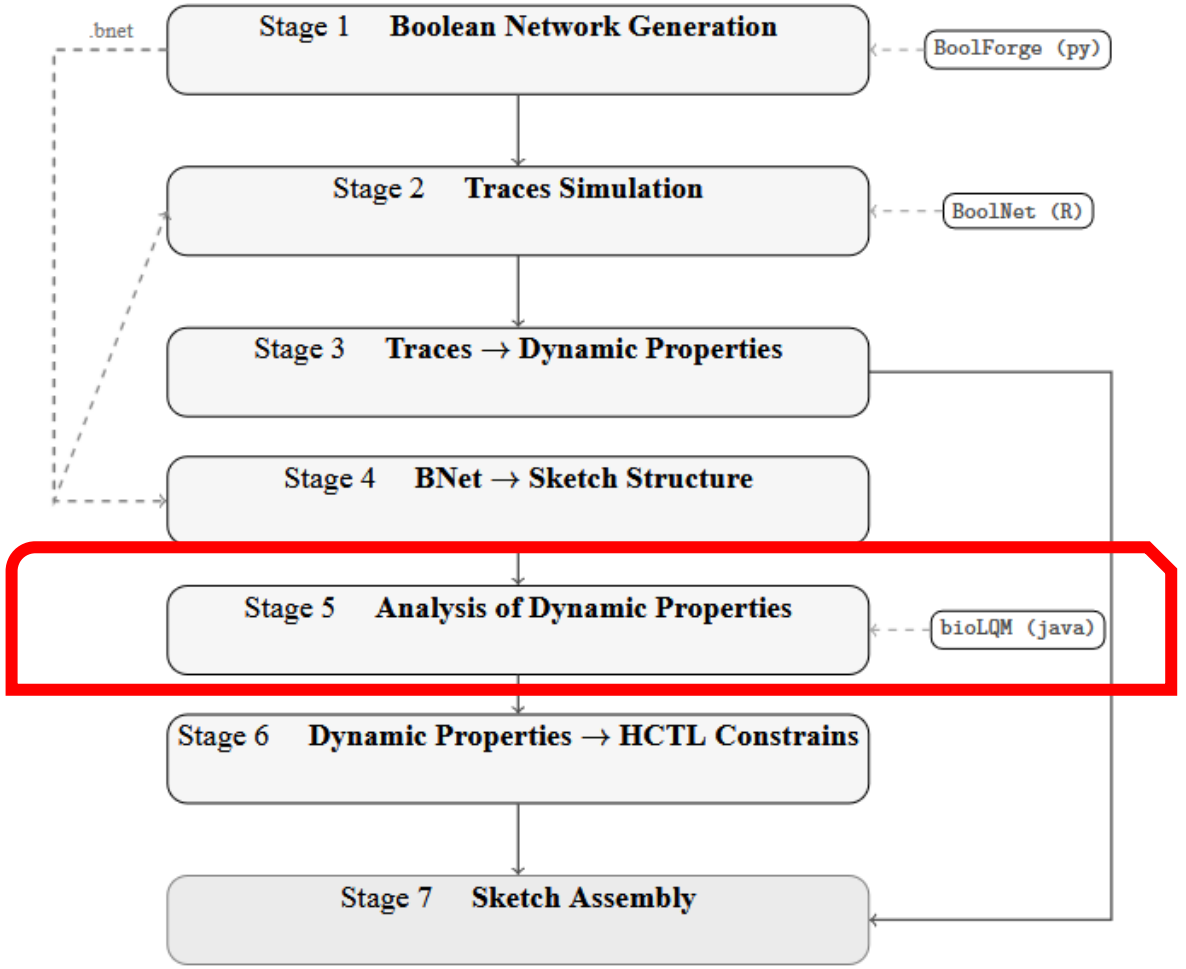
net_fixpoints.txt :

x1 x2 x3 … x20
00001110100111000111
00011111110011000111
10100110101101100100
10110111111001100100

net_trapspaces.txt :

x1 x2 x3 … x20
000-111-1-0-1-000111

'-' means unspecified / any value



3. Sketches Pipeline overview

University of Cyprus

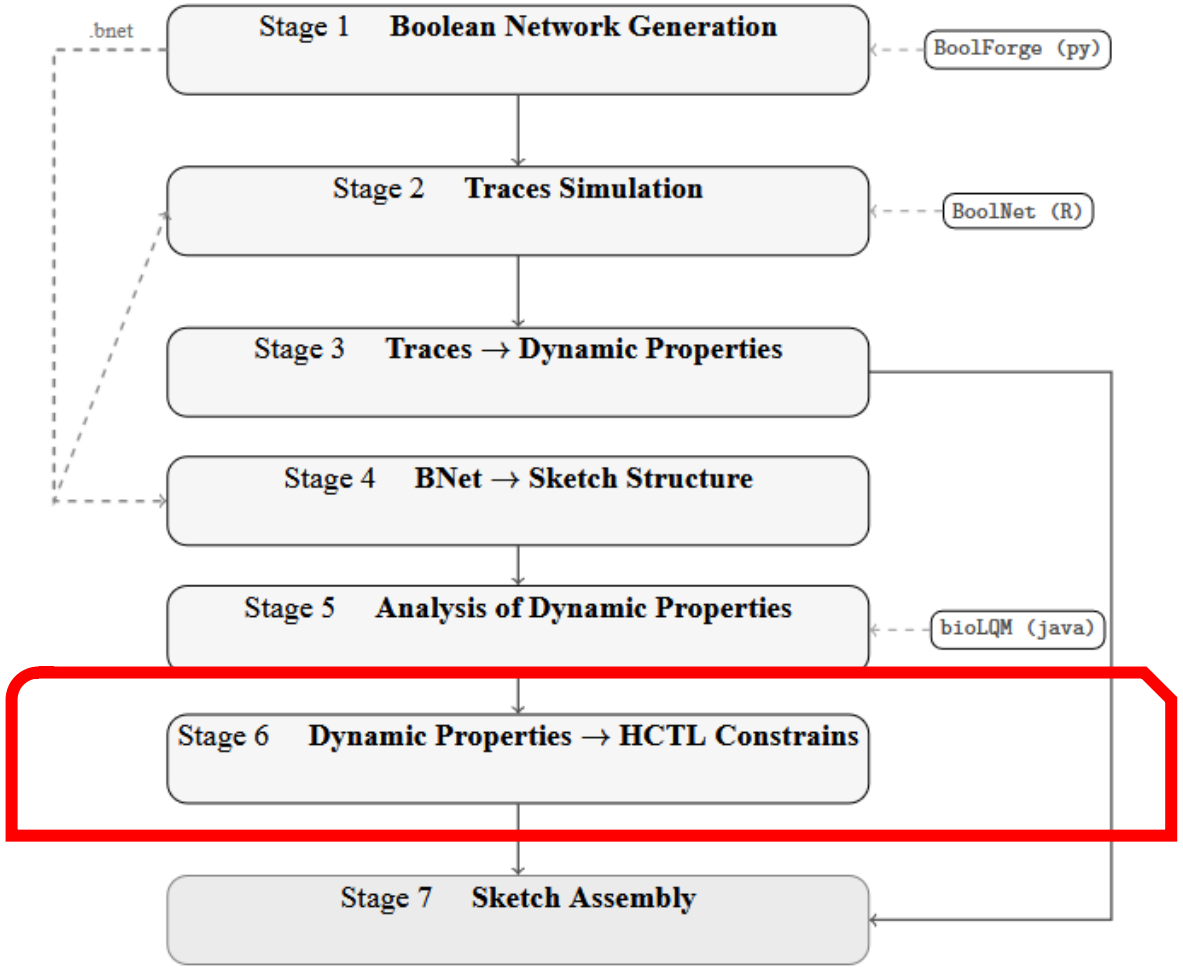
Stage 6 - Boolean Network

INPUT

Bnet from Stage 1 + Stage 6 configuration + net_fixpoints.txt + net_trapspaces.txt

Config parameters

Configuration key	Purpose
`fixpoints`	Path to the fixed-point results produced in Stage 5.
`trapspaces`	Path to the trap-space results produced in Stage 5.
`output`	Path of the generated sketch-property snippet.
`mode`	Selects whether fixed points, trap spaces, or both are converted.
`property_prefix_fixed`	Naming prefix for fixed-point properties.
`property_prefix_trap`	Naming prefix for trap-space properties.
`start_index`	Initial numbering index for generated properties.
`include_forbid_extra`	Controls whether additional exclusion constraints are emitted.
`no_dedup`	Disables duplicate removal.
`no_properties_header`	Omits the property-section header in the output.
`bnet`	Path to the original `.bnet` model used for constant-depth analysis.
`constant_depth`	Controls the depth of clamped-subnetwork analysis. A value of 1 keeps only the original network; 2 adds one-variable clamps; 3 adds two-variable clamps.
`constant_subnetwork_limit`	Maximum number of clamped subnetworks analysed. A value of 0 means no limit.
`constant_property_prefix`	Naming prefix for lifted constant-depth properties.
`biolqm_cmd`	Command used to invoke `bioLQM` for clamped sub-networks.



3. Sketches Pipeline overview

University of Cyprus

Stage 6 - Boolean Network Ge

OUTPUT

HCTL_properties.aeon:

HCTL fixed-points properties:

```
p = (~x1 & ~x2 & ~x3 & ~x4 & x5 & ... & x20)
#!dynamic_property: fixed_point_1:
#`3{x}: ( @{x}: (
#   p
#   & AX(p)
#) )`#
```

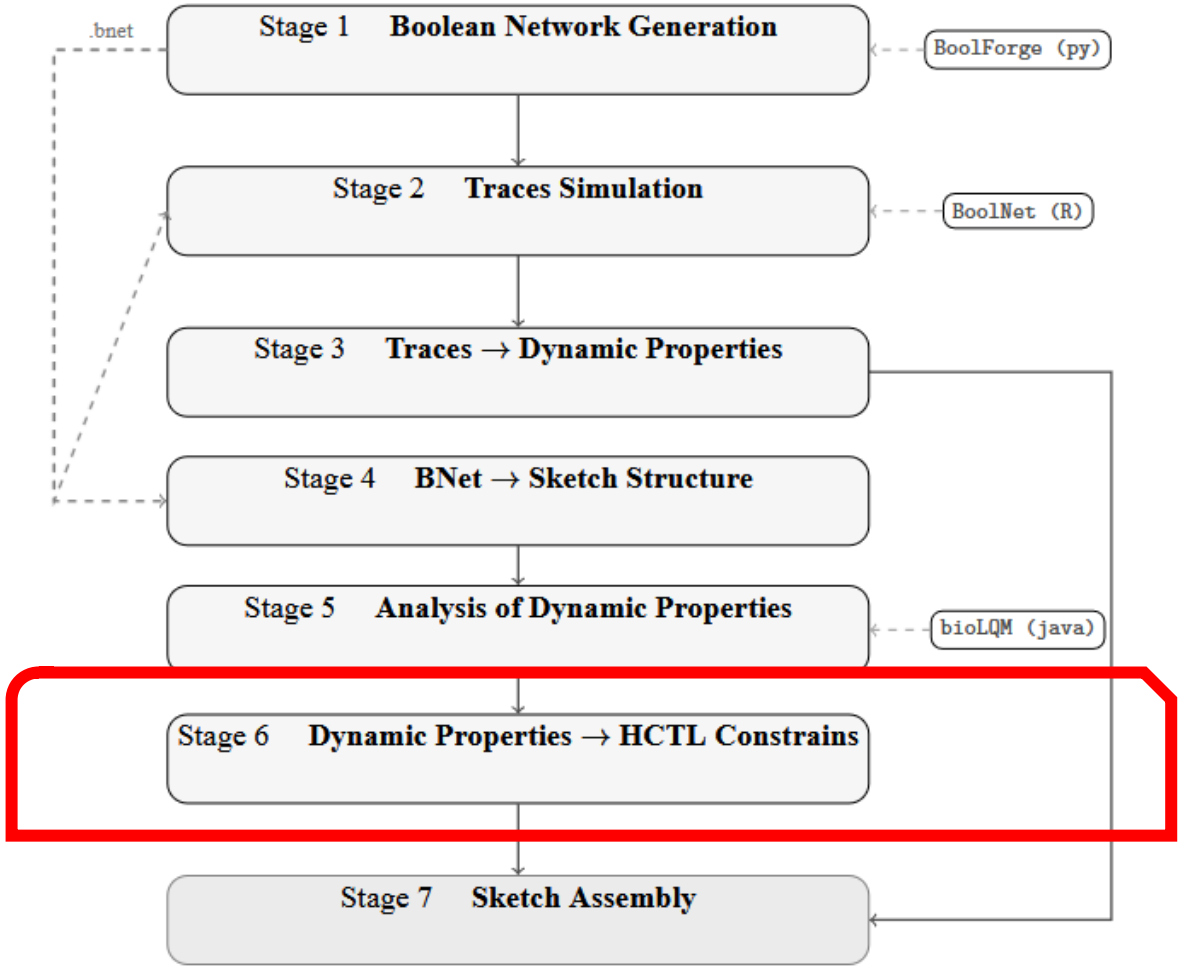
...

```
#!dynamic_property: constant_x1_is_1_fixed_point_1:
#`(3{c}: (@{c}: AG (x1))) =>
#  (3{x}: (@{x}: (
#     (AG (x1)) & p & AX(p)
#  )))`#
```

...

```
#!dynamic_property: constant_x1x2_is_1_fixed_point_1:
#`(3{c}: (@{c}: AG (x1 & x2))) =>
#  (3{x}: (@{x}: (
#     (AG (x1 & x2)) & p & AX(p)
#  )))`#
```

...



3. Sketches Pipeline overview

University of Cyprus

Stage 6 - Boolean Network Ge

OUTPUT

HCTL_properties.aeon:

HCTL trap-space properties:

```
p = (~x1 & ~x2 & x3 & ... & xk ) k<#variables
#! dynamic_property: trap_space_1:
#`3{x}: ( @{x}: (
#   p
#   & AG (p)
#) )`#
```

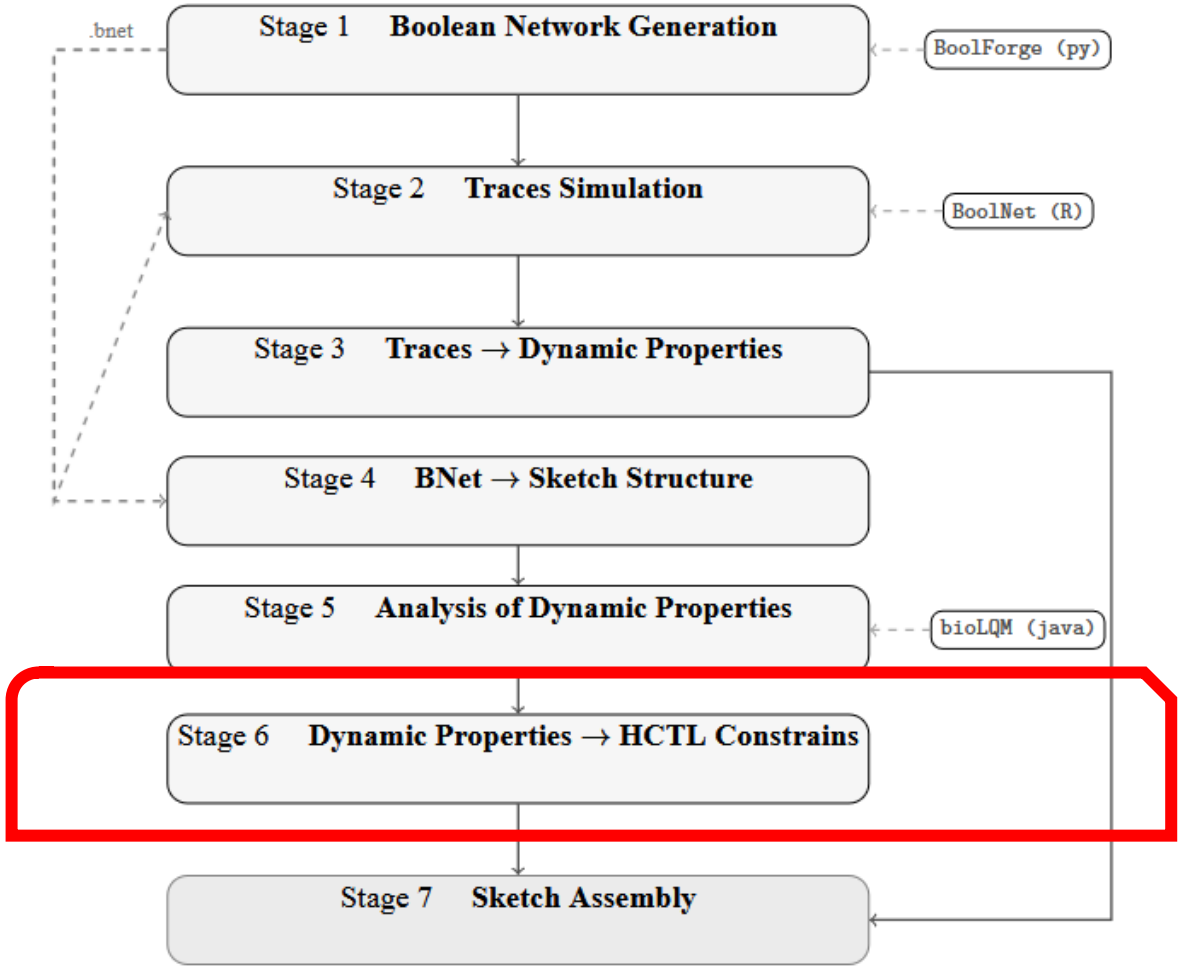
...

```
#!dynamic_property: constant_x1_is_1_trap_space_1:
#`(3{c}: (@{c}: AG (x1))) =>
#  (3{x}: (@{x}: (
#     (AG (x1)) & p & AG(p)
#  )))`#
```

...

```
#!dynamic_property: constant_x1x2_is_1_trap_space_1:
#`(3{c}: (@{c}: AG (x1 & x2))) =>
#  (3{x}: (@{x}: (
#     (AG (x1 & x2)) & p & AG(p)
#  )))`#
```

...



3. Sketches Pipeline overview

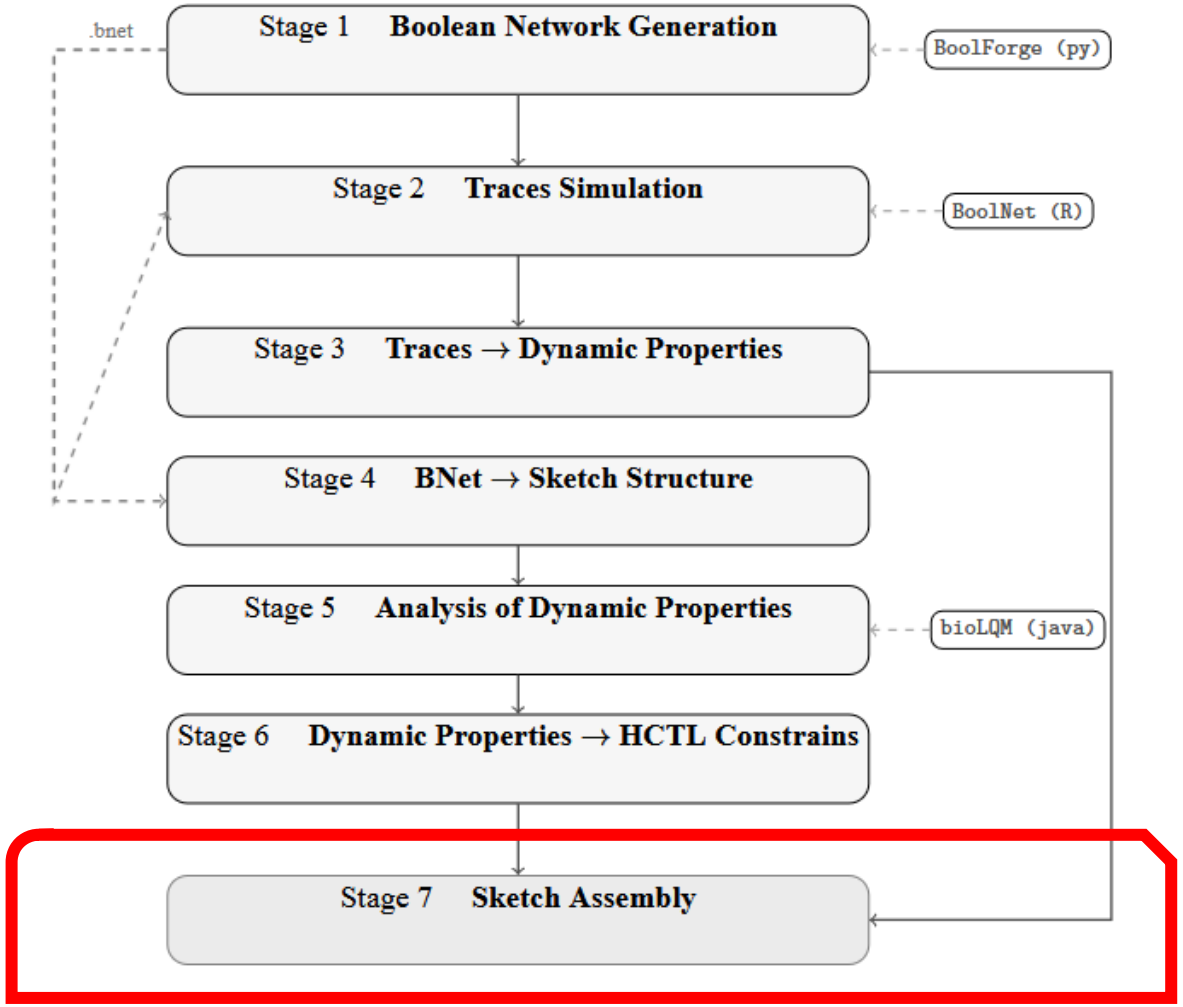
University of Cyprus

Stage 7 – Sketch Assembly

INPUT
All the sketch parts produced from the other stages

Config parameters

Configuration key	Purpose
include_trace_reachability_properties	Includes reachability properties derived from Stage 3.
include_trace_attractor_candidate_properties	Includes trace-based recurrence candidates from Stage 3.
include_trace_cycle_candidate_properties	Includes cycle candidates derived from Stage 3.
include_biolqm_fixed_point_properties	Includes fixed-point properties derived from Stage 6.
include_biolqm_trap_space_properties	Includes trap-space properties derived from Stage 6.
include_essentiality_structure_constraints	Activates essentiality-related structural enrichment in Stage 4.
include_canalization_structure_annotations	Activates canalization-related structural enrichment in Stage 4.



3. Sketches Pipeline overview

Stage 7 – Sketch Assembly

University of Cyprus

MODEL part

```
## MODEL
x10 -> x2
x9 -| x2
$x2:x10 & g_x2(x9)
...
```

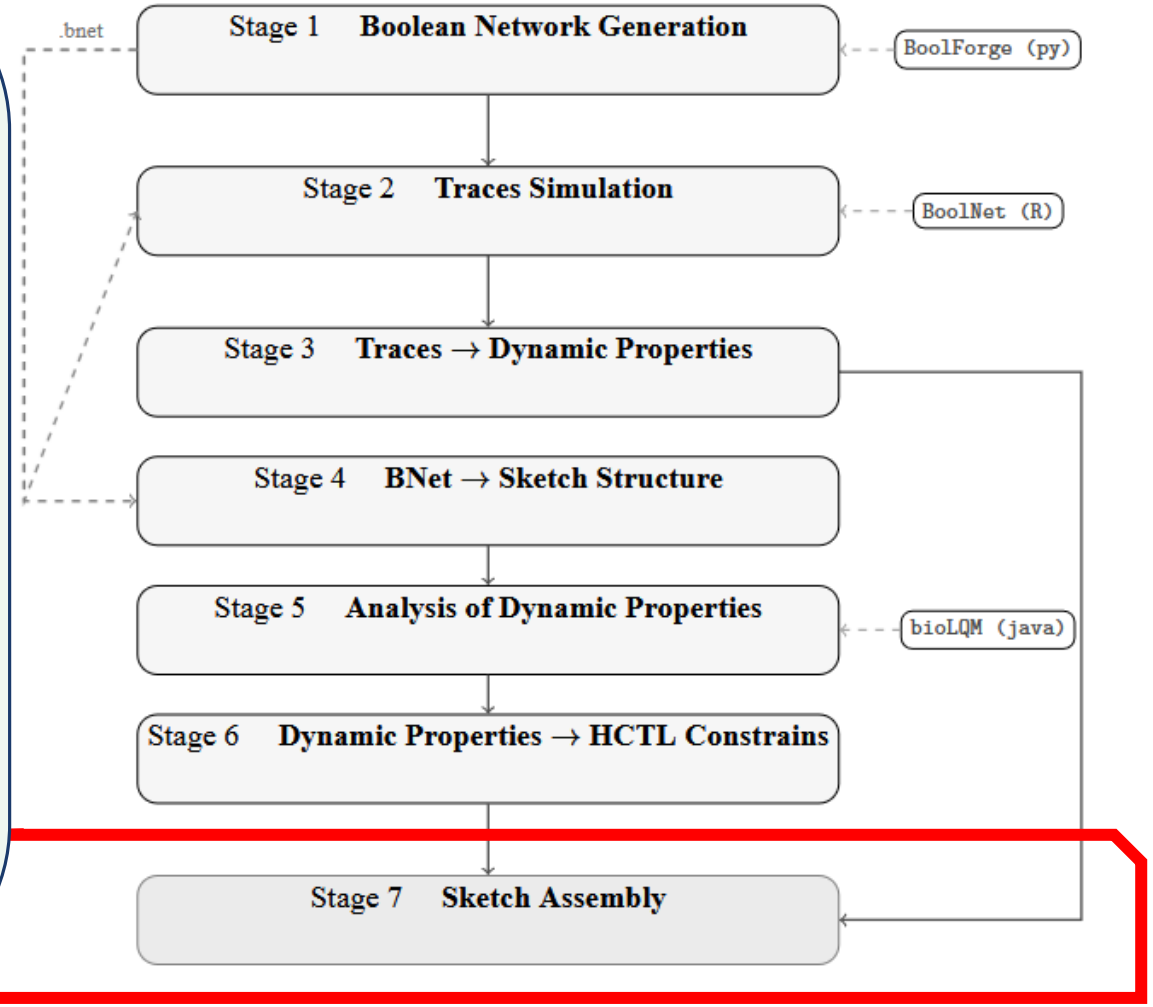
Trace properties

```
## PROPERTIES
#! dynamic_property:
reachability_1:
#`3{x}: @{x}:(p & EF(q))`#
...
```

bioLQM properties

```
## PROPERTIES
#! dynamic_property:
fixed_point_1:
#`3{x}: @{x}:(p & AX(p))`#
...
```

Final .aeon sketch for Sketches inference



4. Sketches Experiment and Evaluation

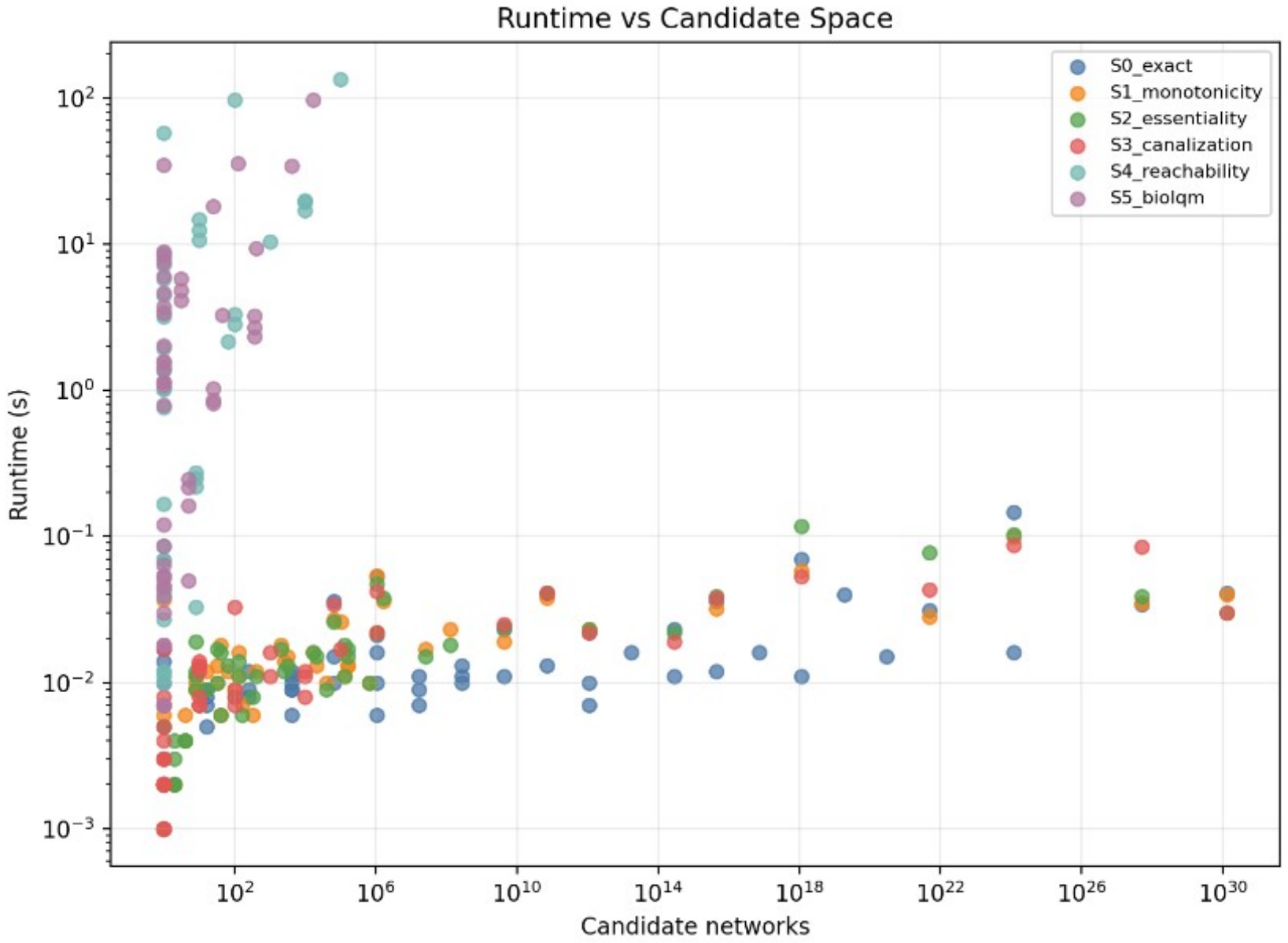
University of Cyprus

Network sizes	Reveal levels	Sketch classes
10	**6**	**6**
5 , 10 , …, 50 variables (steps of 5)	40% → 90% exact functions	S0_exact → S5_biolqm

Total sketches	Metrics recorded	Solver outputs
360	**3**	**360**
6 × 6 × 10 full grid	Runtime · Candidates · Witnesses	Candidate count + up to 10 witnesses

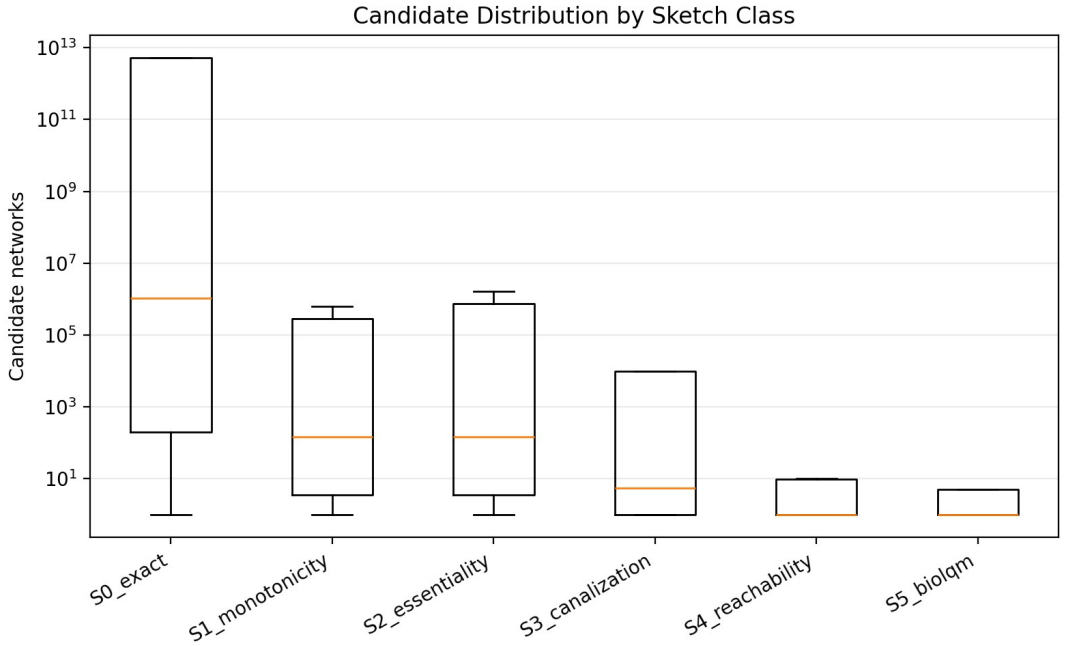
4. Sketches Experiment and Evaluation

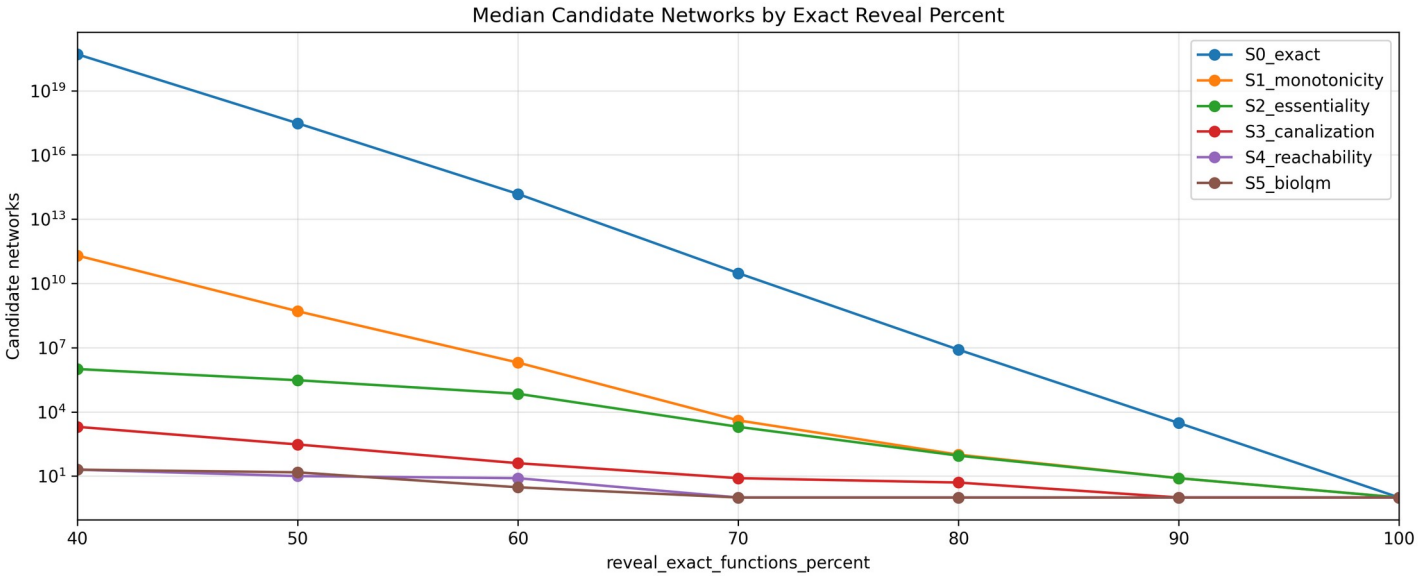
University of Cyprus



4. Sketches Experiment and Evaluation

University of Cyprus





4. Sketches Experiment and Evaluation

University of Cyprus

Sketch class	Information type	Candidate space	Runtime
S0_exact	Partial structure + exact rule revelation	Very large	Low
S1_monotonicity	Signed regulatory constraints	Very large to large	Low
S2_essentiality	Regulator relevance constraints	Very large to large	Low
S3_canalization	Dominant input constraints	Large	Low
S4_reachability	Trace-based dynamic constraints	Small to moderate	High
S5_biolqm	Fixed points and trap spaces	Small	High

University of Cyprus

Conclusion

- Reconstruction becomes difficult for large Boolean networks.
  Observations help, but are not always sufficient.
  Attractors are useful, but computationally expensive.
  Properties of stabilized subnetworks reduce the candidate space.
  Combining structural, functional and dynamic knowledge improves reconstruction.
- The pipeline enables systematic generation and evaluation of synthetic sketches.

University of Cyprus

Future Directions

Reverse engineering from HCTL specifications

Incorporating uncertainty in models and data

Support for probabilistic Boolean networks

More flexible temporal specifications

Extension to multi-valued logical networks

Automatic constraint selection and optimization

- Studying the revision problem in Boolean networks

University of Cyprus

Thank You!

Any questions?



Costas Frantzides

Advisor: Dr Dimopoulos Yiannis
Department of Computer Science, University of Cyprus